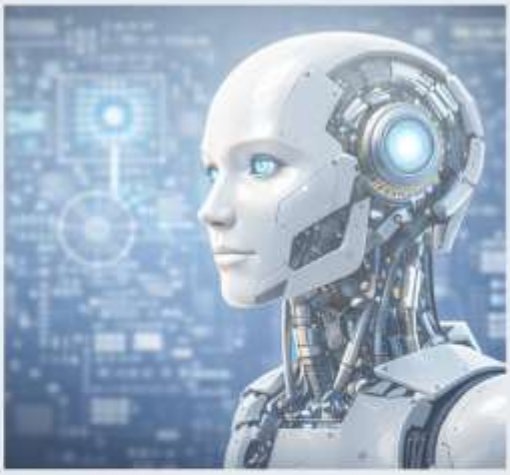
Programming Essentials for ARTIFICIAL INTELLIGENCE



JavaScript Edition: Node.js + Browser + Modern ML/LLM Workflows

A practical mini-eBook focused on building AI systems using JavaScript, without Python.

Author: David Donovan
Released: January 23, 2026

What you'll learn	How to structure AI projects in JavaScript, handle data, run models (TensorFlow.js / ONNX Runtime), and ship reliable LLM apps.
Who it's for	Developers who prefer JavaScript/TypeScript and want to build real AI features for web apps, services, and automation.
What it is not	A math textbook or a framework deep-dive. The goal is practical competence and a solid foundation.

License: You may copy and adapt this document for personal or internal team use.

Table of Contents

1. What "AI Programming" Means in JavaScript
2. Toolchain and Project Workflow (Node, TypeScript, Notebooks)
3. JavaScript Essentials You Actually Use for AI
4. Data Handling: Files, Streams, JSON/CSV, and Schemas
5. Math & Vectors: TypedArrays, Linear Algebra, and Numerical Stability
6. Classic ML in JavaScript (when it's enough)
7. Deep Learning with TensorFlow.js
8. Fast Inference with ONNX Runtime (Node and Web)
9. LLM Apps: APIs, Prompting, Guardrails, and Structured Output
10. Retrieval and Search: Embeddings + Vector Stores
11. Performance, Debugging, and Reliability
12. Deployment Patterns: API, Batch, Edge, and Browser
13. Security and Safety Checklist
14. Practice Projects and a Capstone

Glossary + Next Steps

1. What “AI Programming” Means in JavaScript

AI programming is a pipeline: problem definition, data/inputs, representation, model or rules, evaluation, and shipping with monitoring.

In JavaScript, you typically build AI features in one of three places: (1) a Node.js service, (2) a browser app, or (3) a hybrid where the browser handles UX and the server handles heavy compute or data access.

Your advantage as a programmer is not picking a model. It’s building a system that is measurable, testable, and safe under real user behavior.

> Rule of thumb: treat model outputs as untrusted input. Validate and constrain them the same way you would validate user input.

2. Toolchain and Project Workflow (Node, TypeScript,Notebooks)

Recommended baseline: Node.js 18+ (or 20+), TypeScript, ESLint, Prettier, and a test runner (Vitest or Jest).

For experimentation, you can use plain scripts with small datasets, observable notebooks, or Jupyter with a JavaScript kernel. Promote working experiments into versioned modules.

Use a simple project layout that separates data ingestion, model/inference, evaluation, and serving.

> Non-negotiables: track dataset versions, log evaluation metrics, and keep a reproducible config for any model behavior you ship.

```
# Example layout
ai_js_project/
  data/                  # raw/processed data OR pointers
src/     config.ts     ingest/     features/
inference/     eval/     server/   tests/   scripts/
package.json   tsconfig.json   README.md
```

3. JavaScript Essentials You Actually Use for AI

Most AI code is data transformation. You want clean functions, predictable types, and explicit boundaries.

Prefer TypeScript for anything you’ll maintain. Inference code gets complicated quickly (inputs, outputs, versions, fallbacks).

Master: async/await, error handling, working with streams, JSON schema validation, and performance profiling basics.

Avoid hidden state. AI systems fail quietly; make the state (inputs, versions, thresholds) explicit in code and logs.

```
// Prefer explicit types and pure functions.
export function mean(xs: number[]): number {
  if (xs.length === 0) throw new Error("mean() needs at least one value");
let s = 0;
  for (const x of xs) s += x;
return s / xs.length; }
```

4. Data Handling: Files, Streams, JSON/CSV, andSchemas

Real-world AI work starts with messy data. In Node.js, you'll often parse JSONL, CSV, or pull from a database or object store.

Use streaming I/O for large files; don't read multi-GB datasets into memory. Normalize text and enforce schemas early.

Validation prevents silent corruption: check required fields, types, ranges, and label distributions.

If you can't explain your dataset in one page (fields, sources, leakage risks, known biases), you're not ready to trust the model.

```
import fs from "node:fs"; import
readline from "node:readline";

// JSONL streaming reader
export async function* readJsonl(path: string) {
const rl = readline.createInterface({
    input: fs.createReadStream(path, { encoding: "utf8" }),
crlfDelay: Infinity,
  });
  for await (const line of rl) {
if (!line.trim()) continue;
yield JSON.parse(line);
  }
}

export function normalizeText(s: string): string {
return s.toLowerCase().trim().replace(/\s+/g, " ");
}
```

5. Math & Vectors: TypedArrays, Linear Algebra, andNumerical Stability

Even if you don't implement backprop by hand, you need vector thinking: dot products, cosine similarity, and stable softmax.

In JavaScript, TypedArrays (Float32Array) are key for performance and interoperability with ML runtimes.

Numerical stability matters: subtract max before softmax; clamp probabilities; avoid overflow in exp.

For embeddings and similarity search, store vectors as Float32Array and avoid converting to plain arrays in hot paths.

```
export function dot(a: Float32Array, b: Float32Array): number {
if (a.length !== b.length) throw new Error("Length mismatch");
let s = 0;
  for (let i = 0; i < a.length; i++) s += a[i] * b[i];
return s; }

export function cosineSim(a: Float32Array, b: Float32Array): number {
const ab = dot(a, b);   const aa = Math.sqrt(dot(a, a));   const bb =
Math.sqrt(dot(b, b));   return ab / (aa * bb + 1e-12); }

export function softmax(z: number[]): number[] {
const m = Math.max(...z);
  const exps = z.map(v => Math.exp(v - m));
const sum = exps.reduce((a, b) => a + b, 0);
return exps.map(e => e / sum); }
```

6. Classic ML in JavaScript (when it’s enough)

Not every problem needs deep learning. For many tasks, a rules baseline plus a lightweight model beats complex systems on cost and debuggability.

In JavaScript, you can use the ml.js ecosystem, call a model server, or rely on embeddings + nearest-neighbor retrieval.

Always build a baseline first. If a fancy model does not beat it with clean evaluation, don’t ship the fancy model.

Baselines are not a formality. They are your safety net when models drift or APIs fail.

```
const SPAM_TERMS = ["buy now", "limited offer", "free money"];
export function isSpam(text: string): boolean {
const t = text.toLowerCase();
  return SPAM_TERMS.some(term => t.includes(term));
}
```

7. Deep Learning with TensorFlow.js

TensorFlow.js lets you run and train models in JavaScript (Node or browser). Many teams use it for inference, with training done elsewhere, but you can do both.

Key ideas: tensors have shapes and dtypes; operations are vectorized; you must manage memory in long-running processes.

For production: load a saved model once, validate inputs, batch requests, and guard against latency spikes.

If you see memory climbing in Node, suspect missing tf.dispose() or tensors kept alive by closures.

```
import * as tf from "@tensorflow/tfjs-node"; const model = await

tf.loadLayersModel("file://./model/model.json");

export async function predict(features: number[]): Promise<number[]> {
const x = tf.tensor2d([features]); // shape [1, d]   const y =
model.predict(x) as tf.Tensor;   const out = await y.data();
  tf.dispose([x, y]); // important in services
return Array.from(out); }
```

8. Fast Inference with ONNX Runtime (Node and Web)

ONNX is a common format for exporting models. ONNX Runtime can run these models efficiently in Node.js and in the browser.

This is a great option when you want high-performance inference without pulling in a full training framework.

Treat inference as a pure function: inputs -> outputs, with versioned models and strict validation.

```
import * as ort from "onnxruntime-node"; const session = await

ort.InferenceSession.create("./model.onnx");

export async function infer(input: Float32Array): Promise<Float32Array> {
const tensor = new ort.Tensor("float32", input, [1, input.length]);
const feeds: Record<string, ort.Tensor> = { input: tensor };   const
results = await session.run(feeds);   return results.output.data as
Float32Array;
}
```

9. LLM Apps: APIs, Prompting, Guardrails, andStructured Output

Large language models are best treated as a component in a larger program. Your code should define: allowed actions, required output format, and fallback behavior when the model fails.

Prefer structured output: JSON with a schema. Validate it. Retry on failure. Never execute model output as code.

Prompt injection is real. If the model sees untrusted text (webpages, emails, documents), assume it may contain instructions trying to override your system.

Guardrails live in code: schema validation, permission checks, and hard limits on tool use.

```
type Action = { type: "reply" | "ask_clarify"; message: string };

export function parseAction(s: string): Action | null {
try {
    const obj = JSON.parse(s);
```

```
    if (obj && (obj.type === "reply" || obj.type === "ask_clarify") && typeof obj.message === "strin
return obj;
    }
    return null;
  } catch {
return null;
  }
}
```

10. Retrieval and Search: Embeddings + Vector Stores

Retrieval-Augmented Generation (RAG) grounds LLM answers in your documents.

Pipeline: chunk documents -> embed chunks -> store vectors -> embed query -> retrieve top-k -> generate answer using retrieved text.

In JavaScript, you can compute embeddings via an API or a local runtime, then store vectors in a database designed for similarity search.

> Use citations: when answering, include which chunks were used. This is the fastest way to build trust and debug wrong answers.

```
type Item = { id: string; vec: Float32Array; text: string };

export function topK(query: Float32Array, items: Item[], k = 5): Item[] {
const scored = items.map(it => ({ it, score: cosineSim(query, it.vec) }));
scored.sort((a, b) => b.score - a.score);   return scored.slice(0, k).map(s
=> s.it);
}
```

11. Performance, Debugging, and Reliability

AI systems tend to fail at the boundaries: bad inputs, missing fields, unexpected languages, huge payloads, or latency spikes from upstream APIs.

Build a tiny end-to-end run mode: a small dataset + one request that exercises ingestion, inference, and evaluation in seconds.

Log the right things: model version, thresholds, input stats (not raw sensitive content), latency, and fallback rates.

> If you can’t reproduce a bug from logs, your logs are too weak. Add request IDs, versions, and tight summaries of inputs.

```
export async function timed<T>(name: string, fn: () => Promise<T>): Promise<T> {
const t0 = performance.now();   try {
    return await fn();
  } finally {
    const t1 = performance.now();
    console.log(`${name} ${(t1 - t0).toFixed(1)}ms`);
  }
}
```

12. Deployment Patterns: API, Batch, Edge, andBrowser

Common deployment shapes: batch scoring job, online API inference, edge function for low latency, and browser inference for privacy and responsiveness.

Pick based on constraints: data access, latency, cost, and privacy. Browser inference can avoid sending user data to the server.

Version everything: model artifacts, prompts, retrieval settings, and thresholds. Treat them like code.

> A production AI feature needs monitoring: drift, quality metrics, and error/fallback rates. No monitoring = flying blind.

13. Security and Safety Checklist

Threats: prompt injection, data exfiltration, insecure tool use, training data leaks, and unsafe content generation.

Protections: input validation, output validation, least-privilege API keys, rate limiting, audit logs, and hard-coded allowlists for tools and domains.

If you use an LLM to take actions (send email, write to a database, call external APIs), implement approvals, constraints, and explicit user consent.

```
export function clamp01(x: number): number {
if (!Number.isFinite(x)) return 0;   return
Math.max(0, Math.min(1, x)); }
```

14. Practice Projects and a Capstone

Build these in JavaScript to lock in the skills:

1) Text classifier baseline: rules + embeddings + threshold.

2) Browser image classifier: run a small TF.js model and show top-3 labels with confidence.

3) RAG Q&A; over PDFs: chunk, embed, retrieve, answer with citations.

4) Monitoring simulator: log latency, drift signals, and fallback rates.

Capstone: a customer-support assistant that routes tickets, drafts replies, and cites policy docs — with strict tool allowlists and an approval step.

> Capstone goal: the assistant is useful even when the model is wrong. That means strong retrieval, clear citations, and safe fallbacks.

Glossary + Next Steps

Embedding: vector representation for similarity search.

Inference: running a model to get outputs from inputs.

Drift: real-world inputs change, degrading quality over time.

Calibration: whether predicted probabilities match reality.

Next steps:

- Pick one runtime (TF.js or ONNX Runtime) and build two end-to-end demos.
- Add evaluation scripts and a small labeled dataset.
- Add structured outputs and strict validation for any LLM feature.
- Ship with monitoring and a rollback plan.